GROUP PROJECT

TDW6323 – DATA WRANGLING AND VISUALIZATION

TRIMESTER OCT/NOV 2024 (TERM 2430)

[Group APT]

Submission Date: 6 FEB 2025

NO	STUDENT ID	STUDENT NAME	MAJOR
1.	1201102586	Cheng Wei Ning	BIA
2.	1201102471	Liew Wei Ming	BIA
3.	1201102676	Phang Ao Zan	BIA
4.	1191202283	Uithiswary A/P Velan	BIA

Table of Contents

Introduction 1
Dataset Description 3
Attribute Information: 3
Reason to select this dataset: 4
4 Interesting questions that you would like to know about this dataset: 4
Data Wrangling 5
Exploratory Data Analysis 17
Data Visualization 24
Discussion 34
Question 1 34
Question 2 35
Question 3 37
Question 4 39
Conclusion 41
References 42

Introduction

The fast-growing data landscape demands more strict data organization practices now than ever. The appropriate data needs proper organization for analysis purposes. Most company decisions stem from the analysis of data. The company obtains its data and information through its users and customers. The successful use of raw data depends on essential organizational steps. usable for analytics. In addition, due to the rapid expansion of the amount of data and data Organizing vast amounts of data for analysis has become essential because of the advanced data sources we have today. increasingly necessary. Unstructured raw data exists in a chaotic state which makes it complex for individuals to understand modern data-driven systems. Data driven world. Data wrangling along with data visualization holds exceptional value because this reality has never been clearer.

Data wrangling normally takes place at the very beginning of the data science process. It is the process of removing errors and combining complex data sets to make them more accessible and understandable. Data wrangling is crucial for transforming unstructured data into meaningful analysis. To obtain precise insights and make wise judgements, one needs to be able to clean, combine, and change data. Generally, the task of data wrangling involves scraping raw data from multiple sources, inputting, formatting, transforming, handling missing data, and detecting outliers. However, the exact methods vary from project to project depending upon your data and the objective you are trying to achieve. It is important to process each phase of data wrangling properly since it usually influences the approach used to ensure that the final datasets are reliable and usable.

Apart from that, Data visualization has emerged as a critical component throughout the process of simplifying complex information. The process of converting sophisticated data into simple visual displays has become increasingly important. Data The representation of data through standard graphical elements including charts and plots and infographics and animations constitutes visualization. infographics, and even animations. Data visualization presents information through visual charts and graphs which replace lengthy numerical lists. Data visualization depicts vital information by means of charts and pictures and graphical displays.

These Informational visual representations transform challenging data relationships and data-driven insights into simple to comprehend formats. simple to understand. Visualizations enable organizations to relay their insights effectively to diverse audiences. Data visualization serves two functions: it helps both expert and non-expert stakeholders understand data and it enables the identification of data patterns and trends. a world of data. The quick transmission of data proves difficult without visualization methods. Data visualizations help us identify patterns that enable us to draw insights and achieve better data interaction. In short, the field of data analytics requires indispensable techniques for data wrangling and visualization.

Dataset Description

The dataset that we use is heart disease prediction data (*Heart Disease Prediction Dataset*, n.d.). It contains 1,888 rows and 14 columns, representing medical information for patients. This data is related to heart disease prediction, as it includes attributes such as age, blood pressure, cholesterol levels, and target variables indicating health conditions. The dataset includes both categorical and numerical variables.

Attribute Information:

No	Attribute	Details
1	Age	Age of the patient
2	Sex	Gender of the patient (1 = Male, 0 = Female).
3	CP	Chest pain type (0–3): 0 = typical angina, 1 = atypical angina, etc.
4	Trestbps	Resting blood pressure (in mm Hg).
5	Chol	Serum cholesterol level (in mg/dl).
6	Fbs	Fasting blood sugar > 120 mg/dl (1 = True, 0 = False).
7	Restecg	Resting electrocardiographic results (0, 1, 2).
8	Thalachh	Maximum heart rate achieved.
9	Exang	Exercise-induced angina (1 = Yes, 0 = No).
10	Oldpeak	ST depression induced by exercise relative to rest.
11	Slope	Slope of the peak exercise ST segment (0–2).
12	Ca	Number of major vessels (0–3) colored by fluoroscopy.

13	Thal	Thalassemia (0–3, with categories indicating normal or defect).
14	Target	Target class: presence (1) or absence (0) of heart disease.

Reason to select this dataset:

The reason that we chose this dataset is because cardiovascular diseases (CVDs) stand as the worldwide leader in causing deaths while affecting millions of people each year. Early heart disease identification combined with risk prediction capabilities through healthcare research creates the opportunity for proactive medical treatment. The heart disease prediction dataset enables detailed analysis of essential heart health elements including cholesterol levels, blood pressure and exercise-related symptoms to reveal hidden patterns which traditional observation methods might miss. This data holds the potential to generate novel insights into heart disease risks that could produce substantial progress in disease understanding and risk mitigation. The dataset analysis enables us to support initiatives that decrease mortality rates and enhance treatment results while helping to save lives.

4 Interesting questions that you would like to know about this dataset:

1. What are the most significant factors that contribute to heart disease?
2. How does the maximum heart rate achieved (thalachh) vary across different age groups or genders?
3. Is there a correlation between exercise-induced angina (exang) and other variables like oldpeak or cp?
4. Can a predictive model accurately classify patients as having or not having heart disease based on these attributes?

Data Wrangling

- Import necessary libraries such as pandas, numpy and so on.

[1]:
```
# Import libraries

import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
%matplotlib inline

import seaborn as sns
```

- Load the dataset called “Medicaldataset.csv” using pandas.

[2]:
```
# Read data
data = pd.read_csv("Medicaldataset.csv")
```

- Use .head() to display the first five rows of the dataset.

[3]:
```
data.head()
```

[3]:

	age	sex	cp	trestbps	chol	fbs	restecg	thalachh	exang	oldpeak	slope	ca	thal	target
0	63	1	3	145	233	1	0	150	0	2.3	0	0	1	1
1	37	1	2	130	250	0	1	187	0	3.5	0	0	2	1
2	41	0	1	130	204	0	0	172	0	1.4	2	0	2	1
3	56	1	1	120	236	0	1	178	0	0.8	2	0	2	1
4	57	0	0	120	354	0	1	163	1	0.6	2	0	2	1

- Use .tail() to display the last five rows of the dataset.

[4]:
```
data.tail()
```

[4]:

	age	sex	cp	trestbps	chol	fbs	restecg	thalachh	exang	oldpeak	slope	ca	thal	target
1883	60	1	0	140	207	0	0	138	1	1.9	2	1	3	0
1884	46	1	0	140	311	0	1	120	1	1.8	1	2	3	0
1885	59	1	3	134	204	0	1	162	0	0.8	2	2	2	0
1886	54	1	1	154	232	0	0	164	0	0.0	2	1	2	0
1887	53	1	0	110	335	0	1	143	1	3.0	1	1	3	0

- Display the information of the dataset using .info() such as column name, total number of rows and data type of each column.

[5]:
```
# Display information of the data
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1888 entries, 0 to 1887
Data columns (total 14 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   age       1888 non-null   int64
 1   sex       1888 non-null   int64
 2   cp        1888 non-null   int64
 3   trestbps  1888 non-null   int64
 4   chol      1888 non-null   int64
 5   fbs       1888 non-null   int64
 6   restecg   1888 non-null   int64
 7   thalachh  1888 non-null   int64
 8   exang     1888 non-null   int64
 9   oldpeak   1888 non-null   float64
 10  slope     1888 non-null   int64
 11  ca        1888 non-null   int64
 12  thal      1888 non-null   int64
 13  target    1888 non-null   int64
dtypes: float64(1), int64(13)
memory usage: 206.6 KB
```

- Display the summary statistics of each column.

[6]:
```
# Display statistical value of the data
data.describe().T
```

[6]:

	count	mean	std	min	25%	50%	75%	max
age	1888.0	54.354343	9.081505	29.0	47.75	55.0	61.0	77.0
sex	1888.0	0.688559	0.463205	0.0	0.00	1.0	1.0	1.0
cp	1888.0	1.279131	1.280877	0.0	0.00	1.0	2.0	4.0
trestbps	1888.0	131.549258	17.556985	94.0	120.00	130.0	140.0	200.0
chol	1888.0	246.855403	51.609329	126.0	211.00	241.0	276.0	564.0
fbs	1888.0	0.148305	0.355496	0.0	0.00	0.0	0.0	1.0
restecg	1888.0	0.597458	0.638820	0.0	0.00	1.0	1.0	2.0
thalachh	1888.0	149.424258	23.006153	71.0	133.00	152.0	166.0	202.0
exang	1888.0	0.331568	0.470901	0.0	0.00	0.0	1.0	1.0
oldpeak	1888.0	1.053761	1.161344	0.0	0.00	0.8	1.6	6.2
slope	1888.0	1.421610	0.619588	0.0	1.00	1.0	2.0	3.0
ca	1888.0	0.731462	1.015735	0.0	0.00	0.0	1.0	4.0
thal	1888.0	2.662606	1.249924	0.0	2.00	2.0	3.0	7.0
target	1888.0	0.517479	0.499827	0.0	0.00	1.0	1.0	1.0

- Check if the columns data type is not numeric, convert columns with non-numeric value to numeric values and invalid values are set to NaN.

[7]:
```
# Convert non-numeric columns to numeric

for col in data.columns:
    if not pd.api.types.is_numeric_dtype(data[col]):
        try:
            data[col] = pd.to_numeric(data[col],errors='coerce')
        except ValueError: # Handle cases where conversion is not possible
            print(f"Could not convert column '{col}' to numeric.")
```

[8]:
```
data
```

[8]:

	age	sex	cp	trestbps	chol	fbs	restecg	thalachh	exang	oldpeak	slope	ca	thal	target
0	63	1	3	145	233	1	0	150	0	2.3	0	0	1	1
1	37	1	2	130	250	0	1	187	0	3.5	0	0	2	1
2	41	0	1	130	204	0	0	172	0	1.4	2	0	2	1
3	56	1	1	120	236	0	1	178	0	0.8	2	0	2	1
4	57	0	0	120	354	0	1	163	1	0.6	2	0	2	1
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
1883	60	1	0	140	207	0	0	138	1	1.9	2	1	3	0
1884	46	1	0	140	311	0	1	120	1	1.8	1	2	3	0
1885	59	1	3	134	204	0	1	162	0	0.8	2	2	2	0
1886	54	1	1	154	232	0	0	164	0	0.0	2	1	2	0
1887	53	1	0	110	335	0	1	143	1	3.0	1	1	3	0

1888 rows × 14 columns

- Shows the updated data types for each column after conversion.

[9]:
```
# Display fonal converted data type
data.dtypes
```

[9]:
```
age           int64
sex           int64
cp            int64
trestbps      int64
chol          int64
fbs           int64
restecg       int64
thalachh      int64
exang         int64
oldpeak     float64
slope         int64
ca            int64
thal          int64
target        int64
dtype: object
```

- Check if there are any missing values in each row (True = missing values, False = non-missing values)

```
[10]: # Check for missing value
data.isnull()
```

[10]:

	age	sex	cp	trestbps	chol	fbs	restecg	thalachh	exang	oldpeak	slope	ca	thal	target
0	False	False	False	False	False	False	False	False	False	False	False	False	False	False
1	False	False	False	False	False	False	False	False	False	False	False	False	False	False
2	False	False	False	False	False	False	False	False	False	False	False	False	False	False
3	False	False	False	False	False	False	False	False	False	False	False	False	False	False
4	False	False	False	False	False	False	False	False	False	False	False	False	False	False
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
1883	False	False	False	False	False	False	False	False	False	False	False	False	False	False
1884	False	False	False	False	False	False	False	False	False	False	False	False	False	False
1885	False	False	False	False	False	False	False	False	False	False	False	False	False	False
1886	False	False	False	False	False	False	False	False	False	False	False	False	False	False
1887	False	False	False	False	False	False	False	False	False	False	False	False	False	False

1888 rows × 14 columns

- Extract row and columns containing at least one missing value

```
[11]: # Extract rows and columns where data is missing
missing_values = data[data.isnull().any(axis=1)][data.columns[data.isnull().any(axis=0)]]
print("\nRows and columns with missing values:")
print(missing_values)
```

```
Rows and columns with missing values:
Empty DataFrame
Columns: []
Index: []
```

- Check if there is any missing value with “?” and replace it with NaN.

[12]:
```
# Check if there is any missing values with '?', replace it with NaN
data = data.replace('?', np.nan)

print('\nNumber of instances = %d' % (data.shape[0]))
print('Number of attributes = %d' % (data.shape[1]))

print('Number of misisng values:')
for col in data.columns:
    print('\t%s: %d' % (col,data[col].isna().sum()))
```

```
Number of instances = 1888
Number of attributes = 14
Number of misisng values:
        age: 0
        sex: 0
        cp: 0
        trestbps: 0
        chol: 0
        fbs: 0
        restecg: 0
        thalachh: 0
        exang: 0
        oldpeak: 0
        slope: 0
        ca: 0
        thal: 0
        target: 0
```

- Check if there is any duplicated data in the dataset.

[13]:
```
# Check for duplicated rows
duplicate_rows = data[data.duplicated()]

print(f"Number of duplicate rows: {duplicate_rows.shape[0]}")
print()
if duplicate_rows.shape[0] > 0:
    print("Duplicate Rows:")
    print(duplicate_rows)
else:
    print("No duplicate rows found.")
```

```
Number of duplicate rows: 1286

Duplicate Rows:
      age  sex  cp  trestbps  chol  fbs  restecg  thalachh  exang  oldpeak  \
164    38    1   2       138   175    0        1       173      0      0.0
303    52    1   0       125   212    0        1       168      0      1.0
304    53    1   0       140   203    1        0       155      1      3.1
305    70    1   0       145   174    0        1       125      1      2.6
306    61    1   0       148   203    0        1       161      0      0.0
...   ...  ...  ..       ...   ...  ...      ...       ...    ...      ...
1879   42    1   0       136   315    0        1       125      1      1.8
1881   59    1   2       126   218    1        1       134      0      2.2
1882   40    1   0       152   223    0        1       181      0      0.0
1884   46    1   0       140   311    0        1       120      1      1.8
1885   59    1   3       134   204    0        1       162      0      0.8

      slope  ca  thal  target
164       2   4     2       1
303       2   2     3       0
304       0   0     3       0
305       0   0     3       0
306       2   1     3       0
...     ...  ..   ...     ...
1879      1   0     1       0
1881      1   1     1       0
1882      2   0     3       0
1884      1   2     3       0
1885      2   2     2       0

[1286 rows x 14 columns]
```

- Drop the duplicated rows from the dataset.

[14]:
```
# Drop duplicated rows
data.drop_duplicates(inplace=True)
data
```

[14]:

	age	sex	cp	trestbps	chol	fbs	restecg	thalachh	exang	oldpeak	slope	ca	thal	target
0	63	1	3	145	233	1	0	150	0	2.3	0	0	1	1
1	37	1	2	130	250	0	1	187	0	3.5	0	0	2	1
2	41	0	1	130	204	0	0	172	0	1.4	2	0	2	1
3	56	1	1	120	236	0	1	178	0	0.8	2	0	2	1
4	57	0	0	120	354	0	1	163	1	0.6	2	0	2	1
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
1876	55	1	1	124	261	0	1	141	0	0.3	2	0	3	0
1880	50	1	0	128	204	1	1	156	1	1.0	1	0	0	0
1883	60	1	0	140	207	0	0	138	1	1.9	2	1	3	0
1886	54	1	1	154	232	0	0	164	0	0.0	2	1	2	0
1887	53	1	0	110	335	0	1	143	1	3.0	1	1	3	0

602 rows × 14 columns

- Split column into numerical and categorical features.
- Loops through categorical columns and print their unique values to check for unexpected or invalid entries.

[15]:
```
# Split the data into numerical and categorical
numerical_features = ['age','trestbps','chol','thalachh','oldpeak']
categorical_features = ['sex', 'fbs', 'restecg', 'exang', 'slope', 'ca', 'thal']
```

[16]:
```
# Verify all the categorical features have only expected fills
for col in data[categorical_features].columns:
    print(f'{col}: {data[col].unique()}')
```

```
sex: [1 0]
fbs: [1 0]
restecg: [0 1 2]
exang: [0 1]
slope: [0 2 1 3]
ca: [0 2 1 3 4]
thal: [1 2 3 0 7 6]
```

- Remove rows with unexpected categorical values.

```
[17]: # From the verification, we can noticed that some of the categorical features has some unexpected value
# Therefore, we have to delete the row which has unexpected value
data = data[(data['cp'] <= 3) & (data['ca'] <= 3) & (data['thal'] <= 3) & (data['thal'] >= 1) & (data['slope'] <= 2)]
data
```

[17]:

	age	sex	cp	trestbps	chol	fbs	restecg	thalachh	exang	oldpeak	slope	ca	thal	target
0	63	1	3	145	233	1	0	150	0	2.3	0	0	1	1
1	37	1	2	130	250	0	1	187	0	3.5	0	0	2	1
2	41	0	1	130	204	0	0	172	0	1.4	2	0	2	1
3	56	1	1	120	236	0	1	178	0	0.8	2	0	2	1
4	57	0	0	120	354	0	1	163	1	0.6	2	0	2	1
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
1873	45	1	0	110	275	0	0	118	1	1.0	1	1	2	0
1876	55	1	1	124	261	0	1	141	0	0.3	2	0	3	0
1883	60	1	0	140	207	0	0	138	1	1.9	2	1	3	0
1886	54	1	1	154	232	0	0	164	0	0.0	2	1	2	0
1887	53	1	0	110	335	0	1	143	1	3.0	1	1	3	0

419 rows × 14 columns

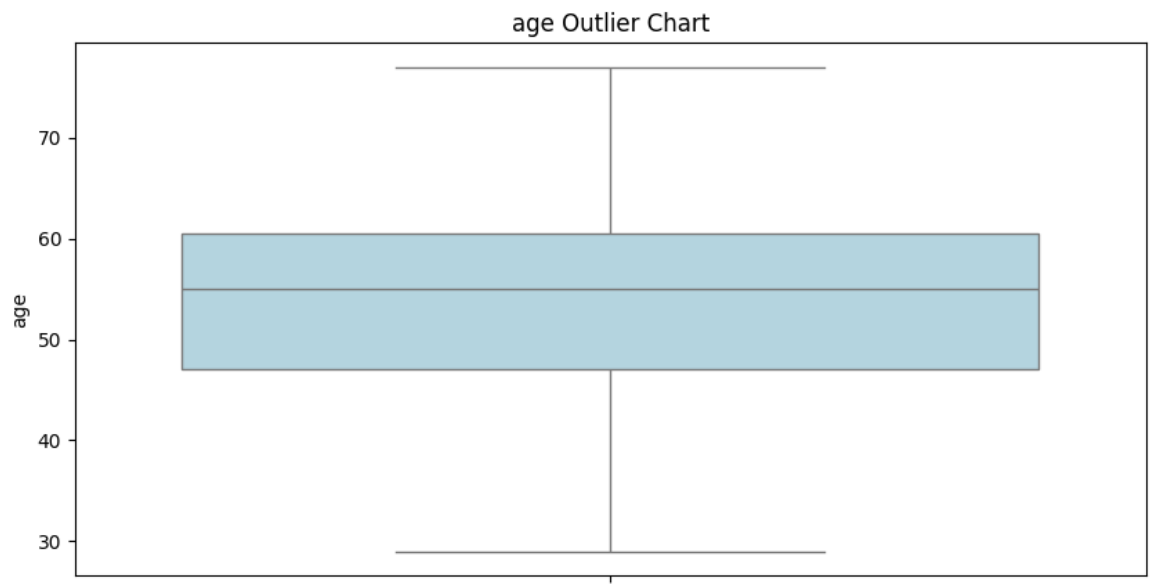
- Loops through numerical features and counts the number of entries equal to 0 (invalid data).
- “oldpeak” does not need to drop as 0 represents ST segment remains unchanged during exercise compared to rest which indicates normal cardiac function or lack of a significant stress-induced ischemia.

```
[18]: # For all numerical value should check if all value is > 0
for col in data[numerical_features].columns:
    print(f'{col}: {(data[col] == 0).sum()}')
```

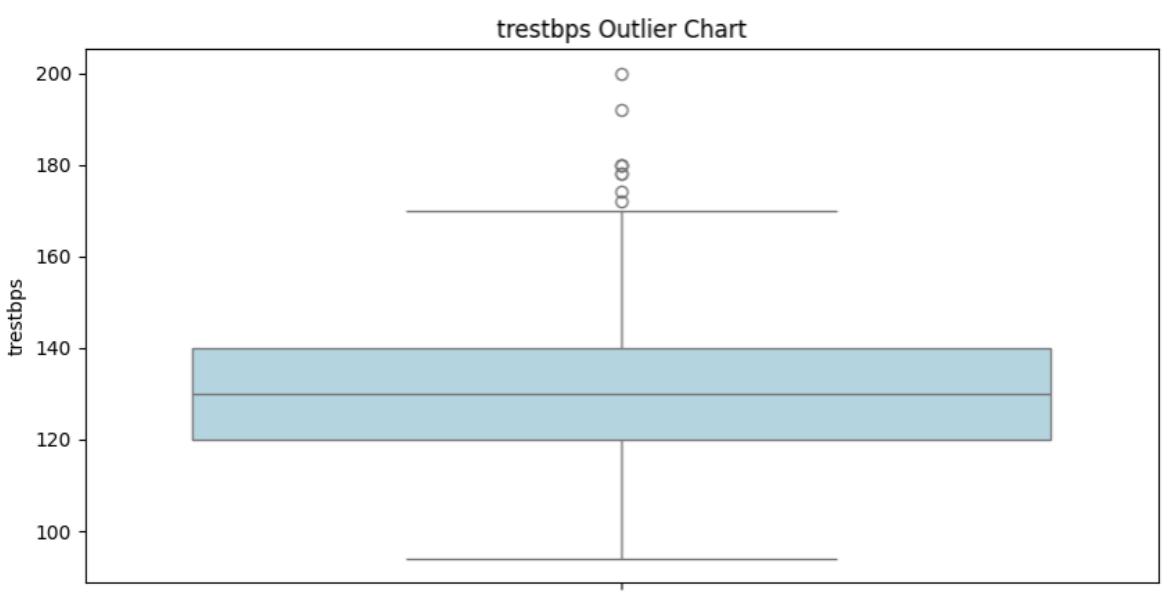
```
age: 0
trestbps: 0
chol: 0
thalachh: 0
oldpeak: 144
```

- Box plot to check for outliers in age, trestbps, chol, thalachh and old peak.
- There is no outlier in age.

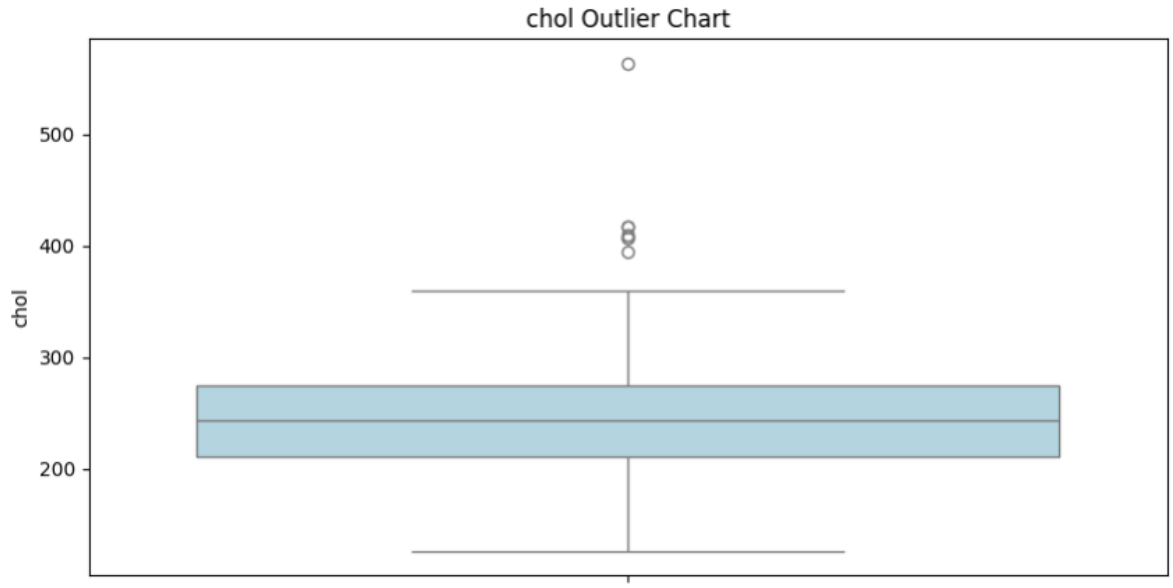
```
[19]: for col in data[numerical_features].columns:
          fig = plt.figure(figsize = (10,5))
          sns.boxplot(data = data, y = col, color = 'LightBlue')  # Boxplot for detecting outliers
          plt.title(f'{col} Outlier Chart')  # Setting the title dynamically for each column
          plt.show()  # Display the plot
```



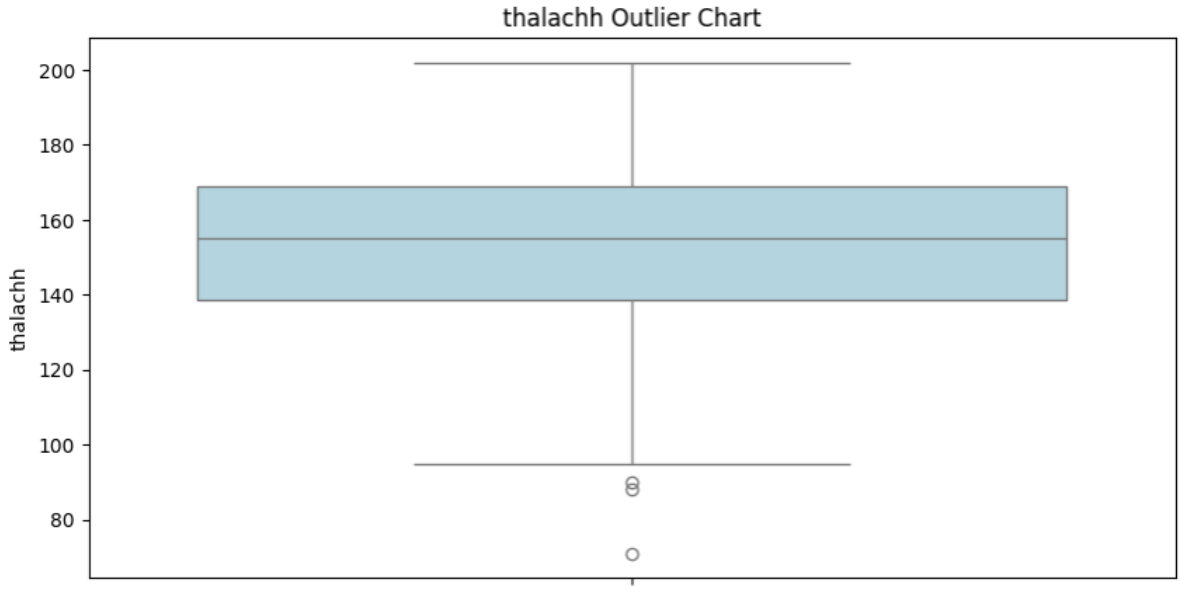
- There are outliers for trestbps, but we do not remove any outliers because it is possible to have a RestingBP of more than 200 mm Hg, so the outliers naturally exist.



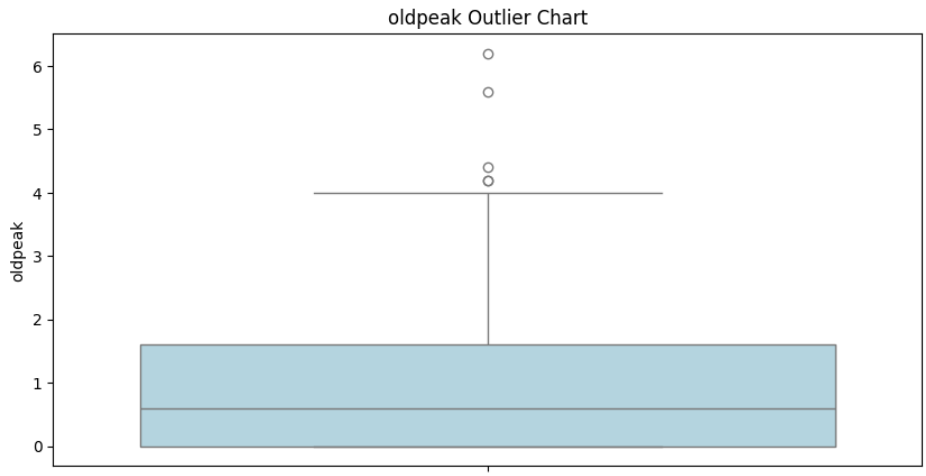
- There are outliers in chol, but we do not remove any outliers because it is possible to have serum cholesterol more than 600 mg/dl, so the outliers naturally exist.



- There are outliers in thalachh, but we do not remove any outliers because it is possible to have a maximum heart rate achieved lower than 80.



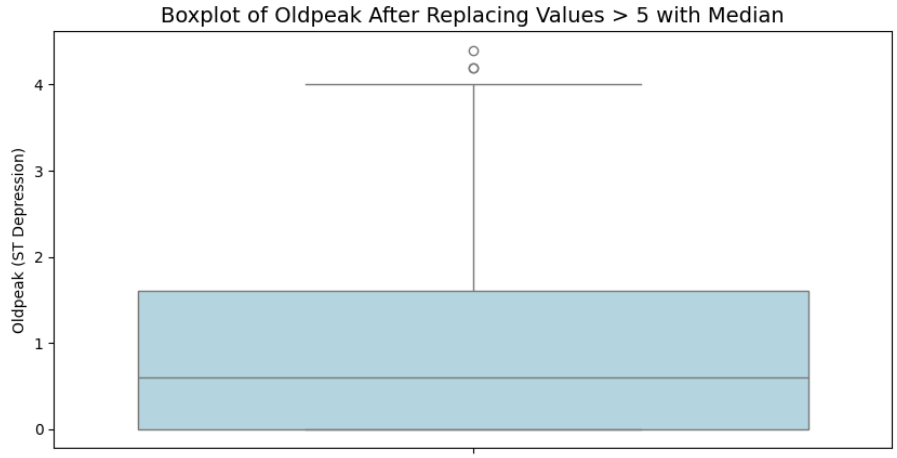
- There are outliers in oldpeak, it is not normal to have ST depression with more than 5mm as most clinical studies and guidelines do not observe or report values this high, if and only if associated with extreme pathological conditions.
- Mild to moderate ST depression (1-2mm) indicated mild ischemia or early signs of reduced blood flow to the heart while severe ST depression (3-4mm) suggests significant myocardial ischemia and possibly severe coronary artery disease.



- Replace oldpeaks that are more than 5mm with median and show the boxplot for verification.

```
[20]: # Replace oldpeak > 5 with the median
data['oldpeak'] = np.where(data['oldpeak'] > 5, data['oldpeak'].median(), data['oldpeak'])

# Create a boxplot to visualize the result
plt.figure(figsize=(10, 5))
sns.boxplot(data=data, y='oldpeak', color='lightblue')
plt.title('Boxplot of Oldpeak After Replacing Values > 5 with Median', fontsize=14)
plt.ylabel('Oldpeak (ST Depression)')
plt.show()
```

```
C:\Users\sabri\AppData\Local\Temp\ipykernel_12348\2433981239.py:2: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning
-a-view-versus-a-copy
  data['oldpeak'] = np.where(data['oldpeak'] > 5, data['oldpeak'].median(), data['oldpeak'])
```



- Reset index after dropping the rows with problems.

[21]:
```
# Reset the index after dropping rows
data.reset_index(drop=True, inplace=True)
data
```

[21]:

	age	sex	cp	trestbps	chol	fbs	restecg	thalachh	exang	oldpeak	slope	ca	thal	target
0	63	1	3	145	233	1	0	150	0	2.3	0	0	1	1
1	37	1	2	130	250	0	1	187	0	3.5	0	0	2	1
2	41	0	1	130	204	0	0	172	0	1.4	2	0	2	1
3	56	1	1	120	236	0	1	178	0	0.8	2	0	2	1
4	57	0	0	120	354	0	1	163	1	0.6	2	0	2	1
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
414	45	1	0	110	275	0	0	118	1	1.0	1	1	2	0
415	55	1	1	124	261	0	1	141	0	0.3	2	0	3	0
416	60	1	0	140	207	0	0	138	1	1.9	2	1	3	0
417	54	1	1	154	232	0	0	164	0	0.0	2	1	2	0
418	53	1	0	110	335	0	1	143	1	3.0	1	1	3	0

419 rows × 14 columns

- Categorize ages and add new column “AgeCategory” into dataset.

[22]:
```
# Function to categorize age
def categorize_age(age):
    if age < 19:
        return 'Young'
    elif age < 36:
        return 'Adult'
    elif age < 56:
        return 'Middle-Aged'
    else:
        return 'Senior'

# Apply the function to create a new column
data['AgeCategory'] = data['age'].apply(categorize_age)
data
```

```
C:\Users\sabri\AppData\Local\Temp\ipykernel_12348\244252572.py:13: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning
-a-view-versus-a-copy
  data['AgeCategory'] = data['age'].apply(categorize_age)
```

[22]:

	age	sex	cp	trestbps	chol	fbs	restecg	thalachh	exang	oldpeak	slope	ca	thal	target	AgeCategory
0	63	1	3	145	233	1	0	150	0	2.3	0	0	1	1	Senior
1	37	1	2	130	250	0	1	187	0	3.5	0	0	2	1	Middle-Aged
2	41	0	1	130	204	0	0	172	0	1.4	2	0	2	1	Middle-Aged
3	56	1	1	120	236	0	1	178	0	0.8	2	0	2	1	Senior
4	57	0	0	120	354	0	1	163	1	0.6	2	0	2	1	Senior
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
414	45	1	0	110	275	0	0	118	1	1.0	1	1	2	0	Middle-Aged
415	55	1	1	124	261	0	1	141	0	0.3	2	0	3	0	Middle-Aged
416	60	1	0	140	207	0	0	138	1	1.9	2	1	3	0	Senior
417	54	1	1	154	232	0	0	164	0	0.0	2	1	2	0	Middle-Aged
418	53	1	0	110	335	0	1	143	1	3.0	1	1	3	0	Middle-Aged

419 rows × 15 columns

- Rearrange AgeCategory column next to age column for easy reference.
- Check the rearrangement with .head().

[23]:
```
# Move column 'AgeCategory' to the next if 'Age'
col = data.pop('AgeCategory')
data.insert(1, 'AgeCategory', col)
```

[24]:
```
data.head()
```

[24]:

	age	AgeCategory	sex	cp	trestbps	chol	fbs	restecg	thalachh	exang	oldpeak	slope	ca	thal	target
0	63	Senior	1	3	145	233	1	0	150	0	2.3	0	0	1	1
1	37	Middle-Aged	1	2	130	250	0	1	187	0	3.5	0	0	2	1
2	41	Middle-Aged	0	1	130	204	0	0	172	0	1.4	2	0	2	1
3	56	Senior	1	1	120	236	0	1	178	0	0.8	2	0	2	1
4	57	Senior	0	0	120	354	0	1	163	1	0.6	2	0	2	1

Exploratory Data Analysis

- Get the information of data.

```
# Display information of the data
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1888 entries, 0 to 1887
Data columns (total 14 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   age       1888 non-null   int64
 1   sex       1888 non-null   int64
 2   cp        1888 non-null   int64
 3   trestbps  1888 non-null   int64
 4   chol      1888 non-null   int64
 5   fbs       1888 non-null   int64
 6   restecg   1888 non-null   int64
 7   thalachh  1888 non-null   int64
 8   exang     1888 non-null   int64
 9   oldpeak   1888 non-null   float64
 10  slope     1888 non-null   int64
 11  ca        1888 non-null   int64
 12  thal      1888 non-null   int64
 13  target    1888 non-null   int64
dtypes: float64(1), int64(13)
memory usage: 206.6 KB
```

```
# Display statistical value of the data
data.describe().T
```

	count	mean	std	min	25%	50%	75%	max
age	1888.0	54.354343	9.081505	29.0	47.75	55.0	61.0	77.0
sex	1888.0	0.688559	0.463205	0.0	0.00	1.0	1.0	1.0
cp	1888.0	1.279131	1.280877	0.0	0.00	1.0	2.0	4.0
trestbps	1888.0	131.549258	17.556985	94.0	120.00	130.0	140.0	200.0
chol	1888.0	246.855403	51.609329	126.0	211.00	241.0	276.0	564.0
fbs	1888.0	0.148305	0.355496	0.0	0.00	0.0	0.0	1.0
restecg	1888.0	0.597458	0.638820	0.0	0.00	1.0	1.0	2.0
thalachh	1888.0	149.424258	23.006153	71.0	133.00	152.0	166.0	202.0
exang	1888.0	0.331568	0.470901	0.0	0.00	0.0	1.0	1.0
oldpeak	1888.0	1.053761	1.161344	0.0	0.00	0.8	1.6	6.2
slope	1888.0	1.421610	0.619588	0.0	1.00	1.0	2.0	3.0
ca	1888.0	0.731462	1.015735	0.0	0.00	0.0	1.0	4.0
thal	1888.0	2.662606	1.249924	0.0	2.00	2.0	3.0	7.0
target	1888.0	0.517479	0.499827	0.0	0.00	1.0	1.0	1.0

```
# Display fonal converted data type
data.dtypes
```

```
age           int64
sex           int64
cp            int64
trestbps      int64
chol          int64
fbs           int64
restecg       int64
thalachh      int64
exang         int64
oldpeak     float64
slope         int64
ca            int64
thal          int64
target        int64
dtype: object
```

- Check for missing values, duplicated data and handle them.

```
# Check for missing value
data.isnull().sum()
```

```
age         0
sex         0
cp          0
trestbps    0
chol        0
fbs         0
restecg     0
thalachh    0
exang       0
oldpeak     0
slope       0
ca          0
thal        0
target      0
dtype: int64
```

```
# To check the duplicated data
data.duplicated().sum()
```

```
1286
```

```
# Drop the duplicated data
data.drop_duplicates(inplace=True)
```

```
# Check the data again
data.duplicated().sum()
```

```
0
```

- Sum of patient, where 1 = Male, 0 = Female.

```
# To check the sum of patient, where 1 = Male, 0 = Female
data['sex'].value_counts()
```

```
sex
1    412
0    190
Name: count, dtype: int64
```

- Sum for Chest pain type (0–3): 0 = typical angina, 1 = atypical angina, etc.

```
# Chest pain type (0-3): 0 = typical angina, 1 = atypical angina, etc.
data['cp'].value_counts()
```

```
cp
0    156
2    131
4    130
3    107
1     78
Name: count, dtype: int64
```

- Sum for Fasting blood sugar > 120 mg/dl (1 = True, 0 = False).

```
# Fasting blood sugar > 120 mg/dl (1 = True, 0 = False).
data['fbs'].value_counts()
```

```
fbs
0    511
1     91
Name: count, dtype: int64
```

- Sum for Resting electrocardiographic results (0, 1, 2).

```
# Resting electrocardiographic results (0, 1, 2).
data['restecg'].value_counts()
```

```
restecg
0    295
1    166
2    141
Name: count, dtype: int64
```

- Sum for Exercise-induced angina (1 = Yes, 0 = No).

```
# Exercise-induced angina (1 = Yes, 0 = No).
data['exang'].value_counts()
```

```
exang
0    399
1    203
Name: count, dtype: int64
```

- Sum for Slope of the peak exercise ST segment (0–2).

```
# Slope of the peak exercise ST segment (0-2).
data['slope'].value_counts()
```

```
slope
1    287
2    273
0     24
3     18
Name: count, dtype: int64
```

- Min and Max Cholesterol by Gender using '.groupby' and '.agg'.

```
# Min and Max Cholesterol by Gender
data.groupby('sex').agg({'chol': ['min', 'max']})
```

	chol	
	min	max
sex		
0	141	564
1	126	407

- Mean of Max Heart Rate by Gender using '.groupby' and '.agg'.

```
# Mean of Max Heart Rate by Gender
data.groupby('sex').agg({'thalachh': ['mean']})
```

	thalachh
	mean
sex	
0	151.678947
1	148.160194

- Mean of Age by Heart Disease Presence using '.groupby' and '.agg'.

```
# Mean of Age by Heart Disease Presence
data.groupby('target').agg({'age': ['mean']})
```

	age
	mean
target	
0	54.758065
1	54.171233

- Count of Heart Disease per Chest Pain Type using '.groupby' and '.agg'.

```
# Count of Heart Disease per Chest Pain Type
data.groupby('cp').agg({'target': ['count']})
```

	target
	count
cp	
0	156
1	78
2	131
3	107
4	130

- Count of Resting ECG per Heart Disease using '.groupby' and '.agg'.

```
# Count of Resting ECG per Heart Disease
data.groupby('restecg').agg({'target': ['count']})
```

	target
	count
restecg	
0	295
1	166
2	141

- Mean of Oldpeak by Gender using '.groupby' and '.agg'.

```
# Mean of Oldpeak by Gender
data.groupby('sex').agg({'oldpeak': ['mean']})
```

	oldpeak
	mean
sex	
0	0.881053
1	1.161893

- Mean of Oldpeak Heart Disease using '.groupby' and '.agg'.

```
# Mean of Oldpeak by Gender
data.groupby('sex').agg({'oldpeak': ['mean']})
```

	oldpeak
	mean
sex	
0	0.881053
1	1.161893

- Mean of Resting Blood Pressure by Chest Pain Type using '.groupby' and '.agg'.

```
# Mean of Resting Blood Pressure by Chest Pain Type
data.groupby('cp').agg({'trestbps': ['mean']})
```

	trestbps
	mean
cp	
0	131.743590
1	132.602564
2	129.862595
3	132.467290
4	132.038462

- Heart Disease Cases by Age using '.groupby' and '.agg'.

```
# Heart Disease Cases by Age
data.groupby(['age', 'target']).agg({'target': ['count']})
```

		target
		count
age	target	
29	0	1
	1	1
34	0	2
	1	2
35	0	3
...	...	...
74	1	1
76	0	1
	1	1
77	0	2
	1	1

82 rows × 1 columns

- Mean of Cholesterol by Resting ECG Type using '.groupby' and '.agg'.

```
# Mean of Cholesterol by Resting ECG Type
data.groupby('restecg').agg({'chol': ['mean']})
```

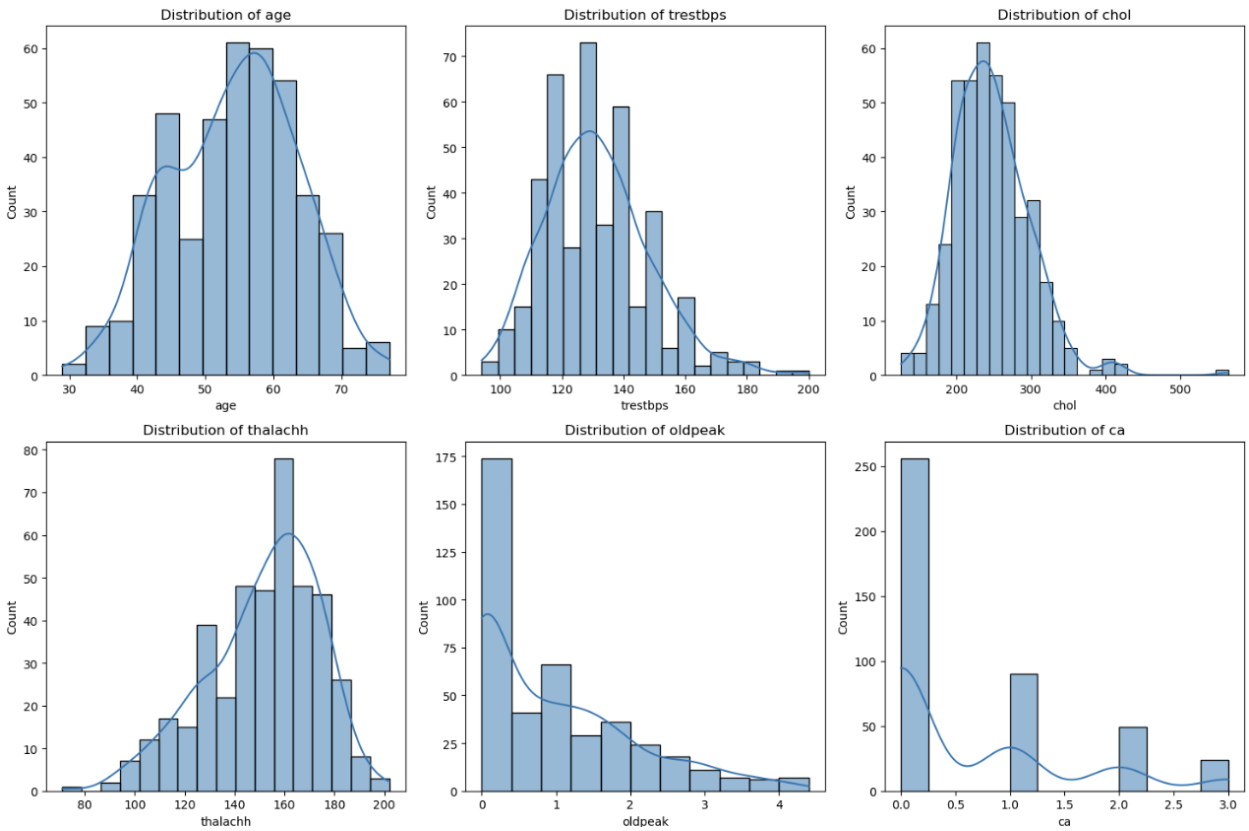
	chol
restecg	mean
0	248.894915
1	239.409639
2	258.163121

Data Visualization

- Histplot to plot the graph of all numerical features.
- Visualize the distribution for all numerical features which are age, trestbps, chol, thalachh, oldpeak and ca.

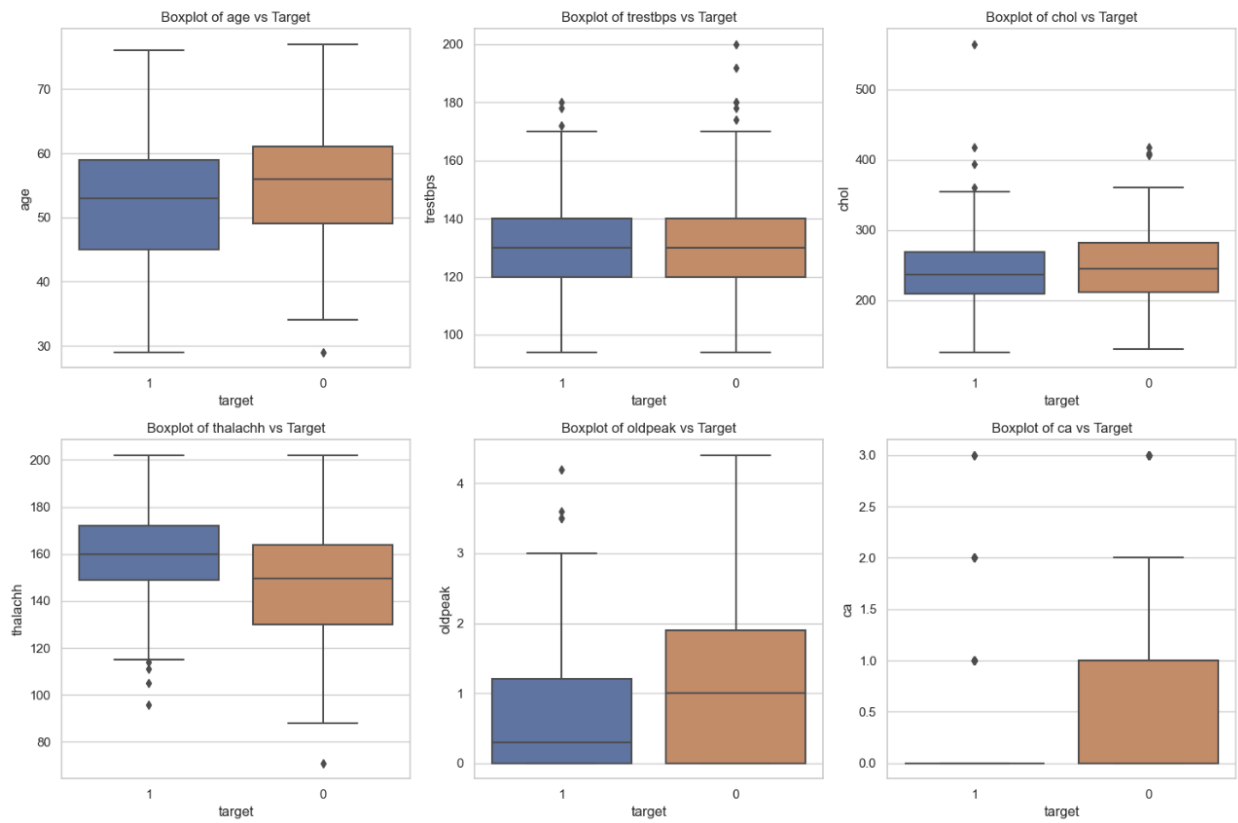
```
data['target'] = data['target'].astype(str)
```

```
# Plotting numerical features
numerical_features = ['age', 'trestbps', 'chol', 'thalachh', 'oldpeak', 'ca']

plt.figure(figsize=(15, 10))
for i, col in enumerate(numerical_features):
    plt.subplot(2, 3, i + 1)
    sns.histplot(data[col], kde=True)
    plt.title(f'Distribution of {col}')

plt.tight_layout()
plt.show()
```



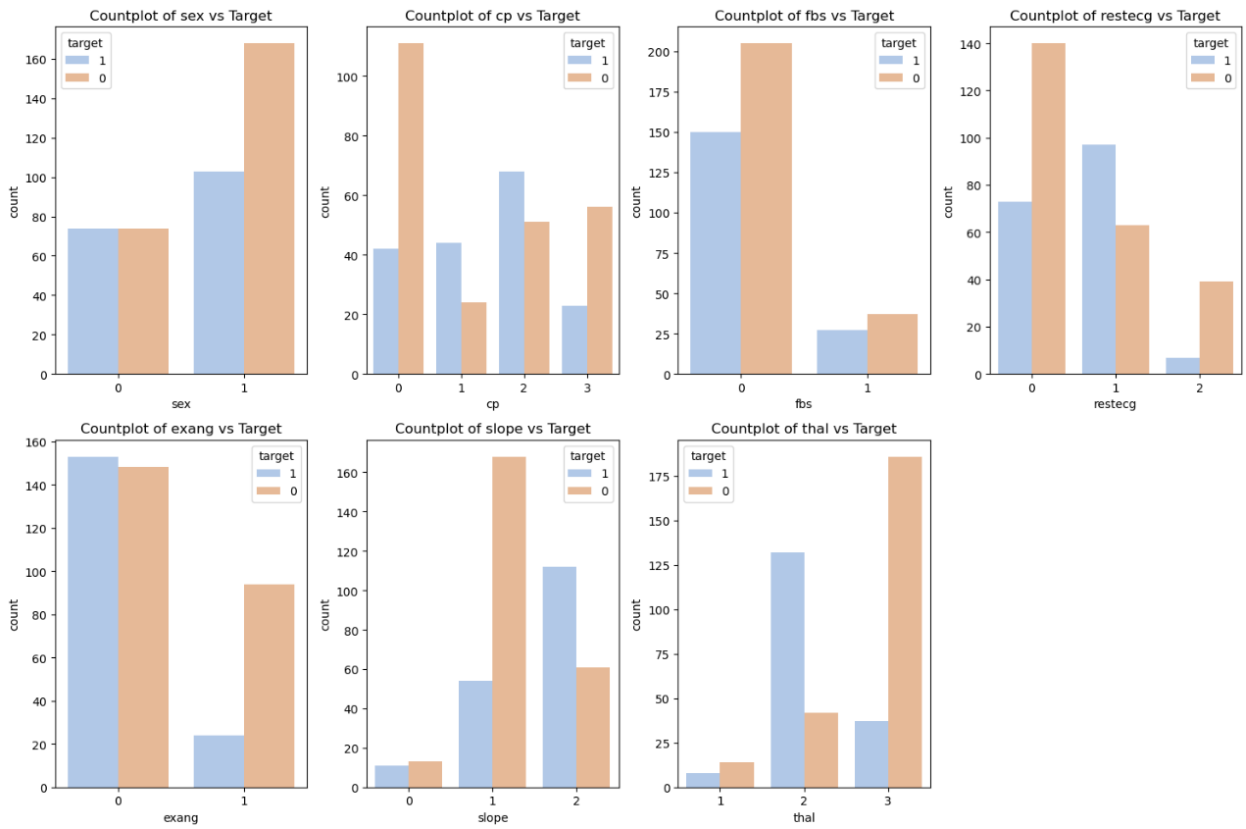
- Boxplot to plot the graph of all numerical features.
- Visualize the median, interquartile range, the whiskers and outliers for all numerical features against the target attribute.

```
plt.figure(figsize=(15, 10))
for i, col in enumerate(numerical_features):
  plt.subplot(2, 3, i + 1)
  sns.boxplot(x='target', y=col, data=data)
  plt.title(f'Boxplot of {col} vs Target')
plt.tight_layout()
plt.show()
```



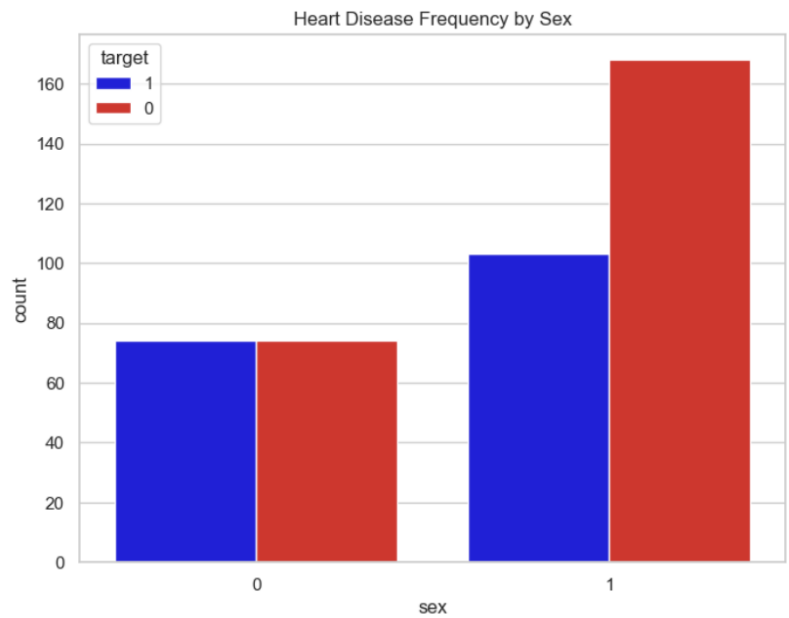
- Countplot to plot the graph of all categorical features.
- Visualize the count of all categorical features against target attribute.
- The categorical features are sex, cp, fbs, restecg, exang, slope and thal.

```
# Plotting categorical features
categorical_features = ['sex', 'cp', 'fbs', 'restecg', 'exang', 'slope', 'thal']

plt.figure(figsize=(15, 10))
for i, col in enumerate(categorical_features):
    plt.subplot(2, 4, i + 1)
    sns.countplot(x=col, data=data, hue='target', palette='pastel')
    plt.title(f'Countplot of {col} vs Target')

plt.tight_layout()
plt.show()
```



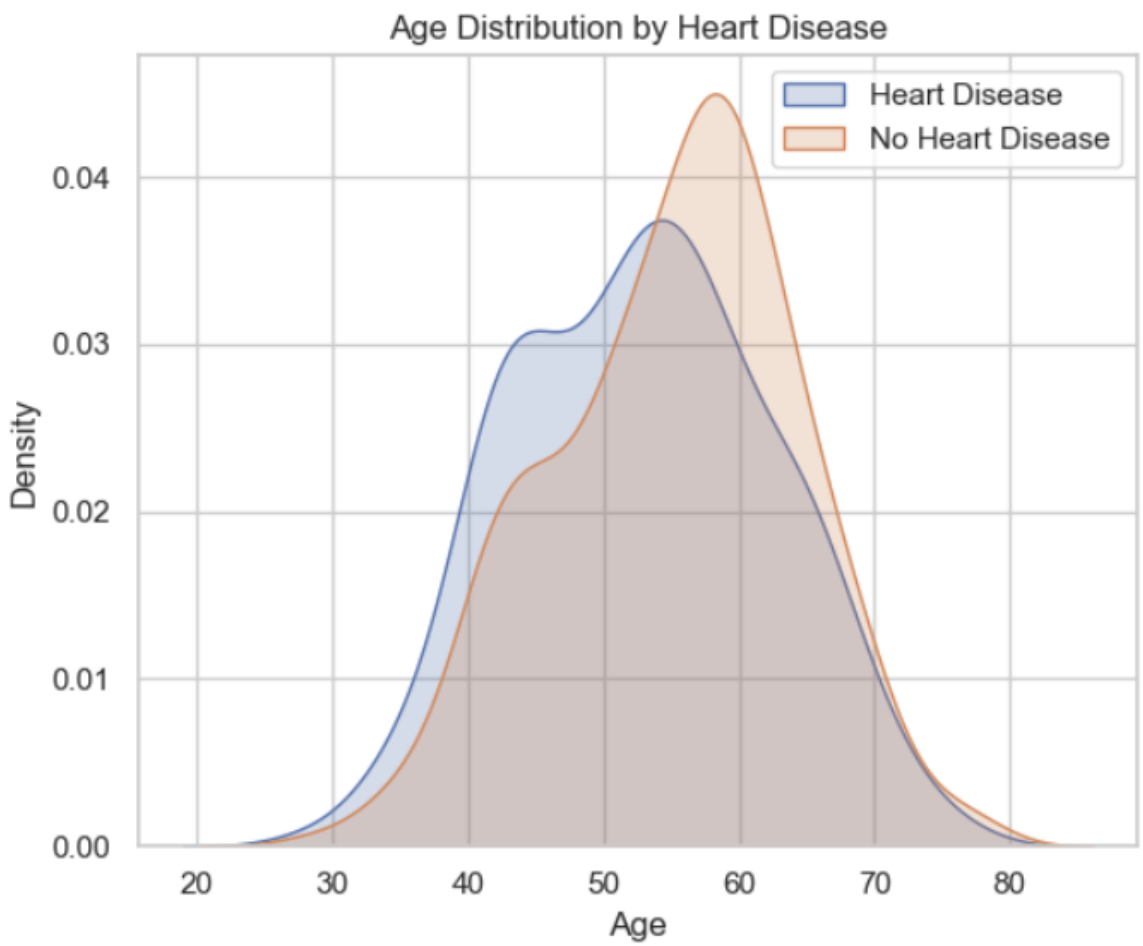
- Countplot to plot heart disease frequency by sex.
- Blue shows individuals have heart disease and orange shows have no heart diseases based on their sex where 0 is Female and 1 is Male.
- Males tend to get heart disease than female.

```
# Heart Disease Frequency by Sex
plt.figure(figsize=(8, 6))
sns.countplot(x='sex', hue='target', data=data, palette=["blue", "red"])
plt.title('Heart Disease Frequency by Sex')
plt.show()
```



- Kdeplot to plot on age distribution by target whether have heart disease or no heart disease.
- Blue shows individuals have heart disease and red shows have no heart diseases based on their age.
- Individuals around age of 50 to 60 may face heart disease.

```
plt.figure(figsize=(6, 5))
sns.kdeplot(data.loc[data['target'] == '1', 'age'], label='Heart Disease', fill=True)
sns.kdeplot(data.loc[data['target'] == '0', 'age'], label='No Heart Disease', fill=True)
plt.xlabel('Age')
plt.ylabel('Density')
plt.title('Age Distribution by Heart Disease')
plt.legend()
plt.tight_layout()
plt.show()
```

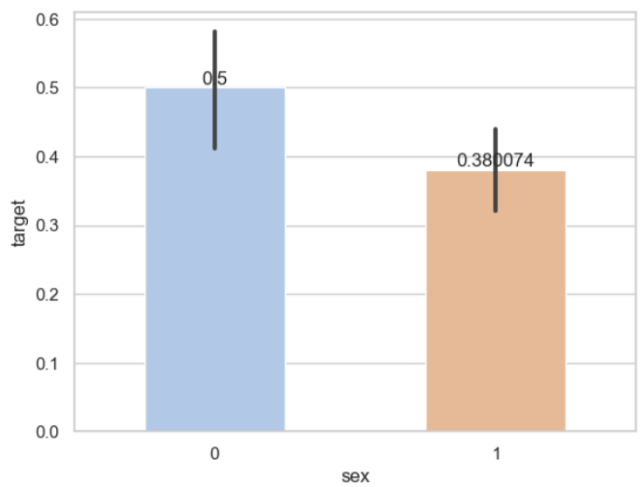


- Barplot show the mean of male (1) and female (0) suffer from heart disease.

- Male (0) have the mean of 0.5 in having heart disease than female (1) with a mean of 0.38.

```
data['target'] = data['target'].astype(int)
```

```
# the mean of each Heart Disease
sns.set(style="whitegrid")
ax = sns.barplot(x="sex", y="target", data=data, estimator= mean, width=0.5, palette='pastel')

for label in ax.containers:
    ax.bar_label(label)
```

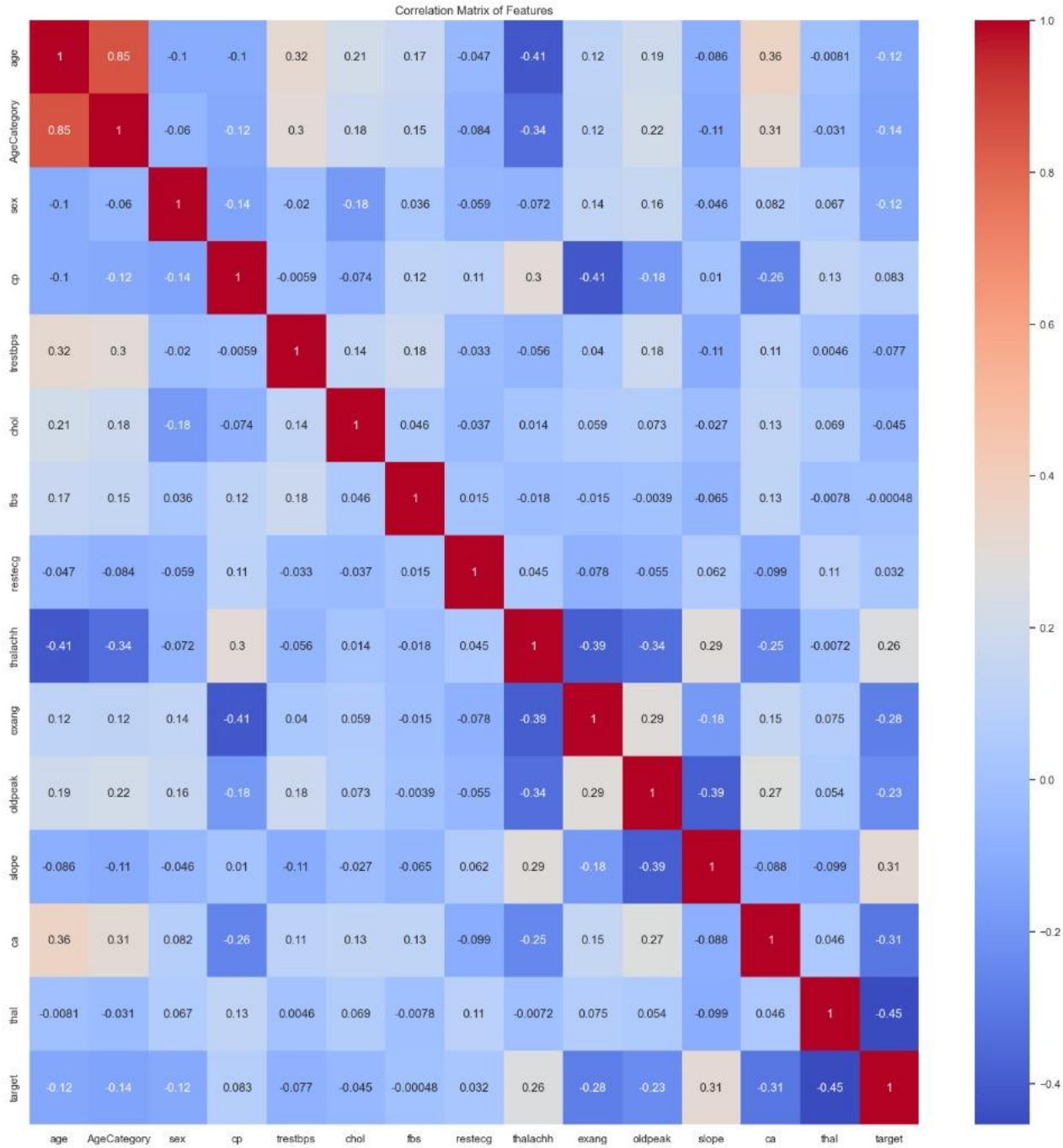


- Heatmap on correlation Matrix of all features.
- Visualize the correlation between independent and dependent variables.

```
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()
data['AgeCategory'] = le.fit_transform(data['AgeCategory'])
```

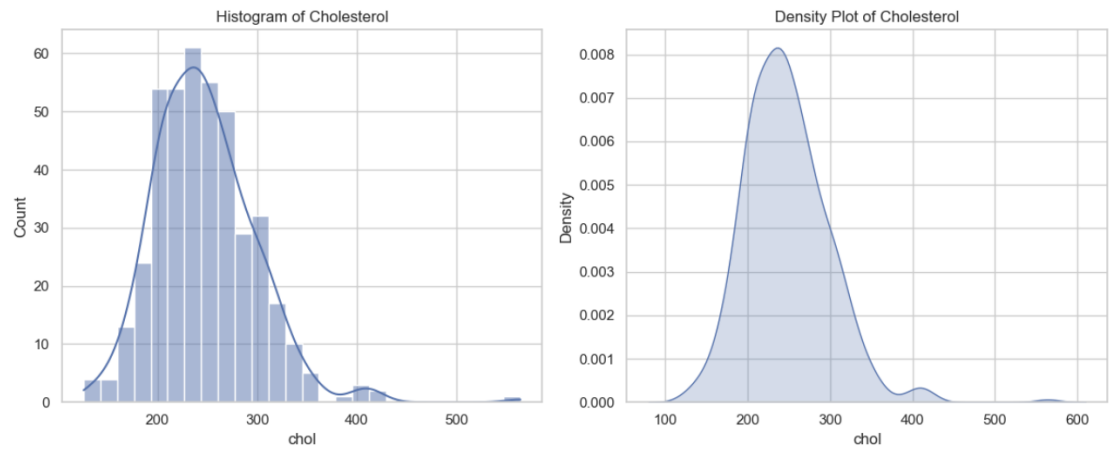
```
data.corr()
```

	age	AgeCategory	sex	cp	trestbps	chol	fbs	restecg	thalachh	exang	oldpeak	slope	ca	thal
age	1.000000	0.850486	-0.103959	-0.102658	0.320511	0.213225	0.167438	-0.046681	-0.410052	0.120568	0.193913	-0.086238	0.358404	-0.008111
AgeCategory	0.850486	1.000000	-0.060129	-0.116407	0.299561	0.179266	0.148822	-0.084048	-0.338749	0.116807	0.215984	-0.110427	0.310993	-0.030638
sex	-0.103959	-0.060129	1.000000	-0.135818	-0.020388	-0.180169	0.036175	-0.058855	-0.071581	0.140767	0.164758	-0.045736	0.082167	0.067062
cp	-0.102658	-0.116407	-0.135818	1.000000	-0.005948	-0.074392	0.121803	0.108752	0.300397	-0.411333	-0.182578	0.010308	-0.260621	0.134575
trestbps	0.320511	0.299561	-0.020388	-0.005948	1.000000	0.135577	0.178672	-0.032657	-0.056192	0.039652	0.178082	-0.105293	0.110229	0.004588
chol	0.213225	0.179266	-0.180169	-0.074392	0.135577	1.000000	0.046138	-0.036622	0.014136	0.058602	0.073352	-0.026834	0.133385	0.069277
fbs	0.167438	0.148822	0.036175	0.121803	0.178672	0.046138	1.000000	0.014766	-0.018121	-0.015102	-0.003942	-0.065169	0.127188	-0.007817
restecg	-0.046681	-0.084048	-0.058855	0.108752	-0.032657	-0.036622	0.014766	1.000000	0.045133	-0.078052	-0.055173	0.062391	-0.099095	0.107114
thalachh	-0.410052	-0.338749	-0.071581	0.300397	-0.056192	0.014136	-0.018121	0.045133	1.000000	-0.392103	-0.339531	0.287512	-0.254227	-0.007171
exang	0.120568	0.116807	0.140767	-0.411333	0.039652	0.058602	-0.015102	-0.078052	-0.392103	1.000000	0.288428	-0.180667	0.151695	0.074794
oldpeak	0.193913	0.215984	0.164758	-0.182578	0.178082	0.073352	-0.003942	-0.055173	-0.339531	0.288428	1.000000	-0.389252	0.270319	0.054349
slope	-0.086238	-0.110427	-0.045736	0.010308	-0.105293	-0.026834	-0.065169	0.062391	0.287512	-0.180667	-0.389252	1.000000	-0.087849	-0.098971
ca	0.358404	0.310993	0.082167	-0.260621	0.110229	0.133385	0.127188	-0.099095	-0.254227	0.151695	0.270319	-0.087849	1.000000	0.045669
thal	-0.008111	-0.030638	0.067062	0.134575	0.004588	0.069277	-0.007817	0.107114	-0.007171	0.074794	0.054349	-0.098971	0.045669	1.000000
target	-0.124984	-0.136037	-0.116047	0.082637	-0.076885	-0.045482	-0.000481	0.032415	0.262284	-0.277658	-0.233082	0.313652	-0.309903	-0.453649


Correlation Matrix of Features
age
AgeCategory
sex
cp
trestbps
chol
fbs
restecg
thalachh
exang
oldpeak
slope
ca
thal
target
1.0
0.8
0.6
0.4
0.2
0.0
−0.2
−0.4

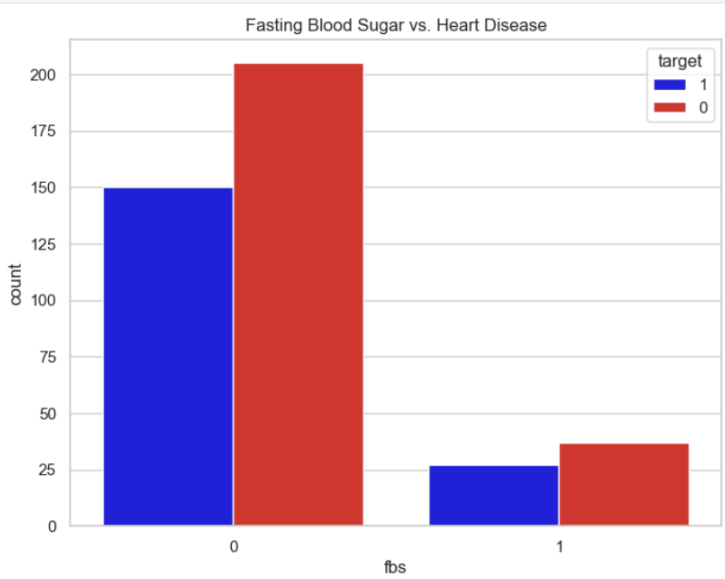
- Histplot on distribution of cholesterol.
- Kdeplot on density of cholesterol.

```
# plot the graph of cholesterol
fig, ax = plt.subplots(1, 2, figsize=(12, 5))
sns.histplot(data['chol'], kde=True, ax=ax[0])
ax[0].set_title('Histogram of Cholesterol')
sns.kdeplot(data['chol'], ax=ax[1],fill=True)
ax[1].set_title('Density Plot of Cholesterol')
plt.tight_layout()
plt.show()
```



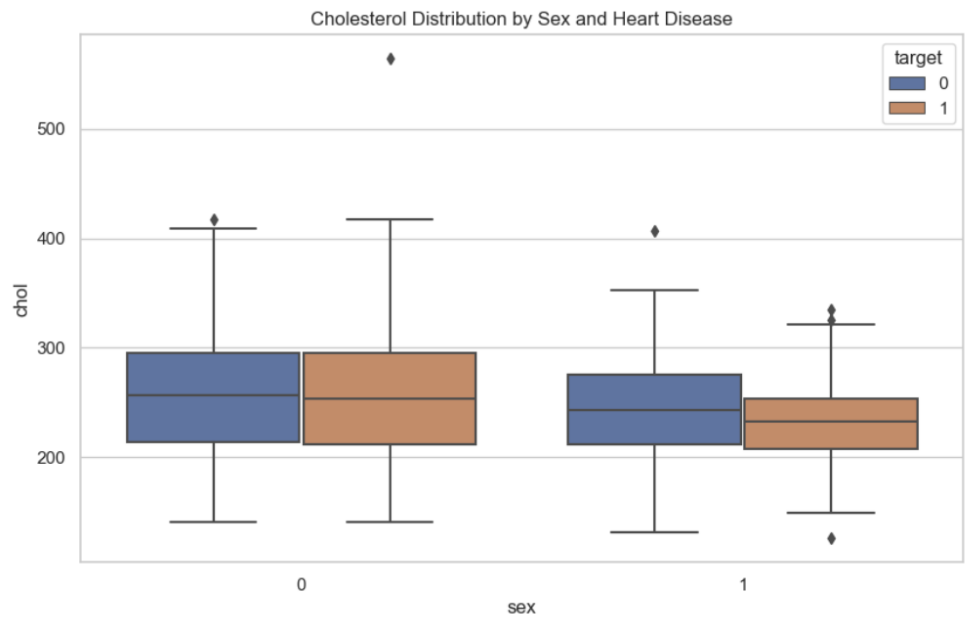
- Countplot on Fasting Blood Sugar against Heart disease.
- Fbs 1 is true where fbs > 120mg/dl whereas fbs 0 is false.
- Blue shows individuals have heart disease and red shows have no heart diseases.

```
# Graph of Fastings and HeartDisease
plt.figure(figsize=(8, 6))
sns.countplot(x='fbs', hue='target', data=data, palette=['blue', 'red'])
plt.title('Fasting Blood Sugar vs. Heart Disease')
plt.show()
```



- Boxplot on Cholesterol distribution by sex and heart disease.
- Visualize the median, interquartile range, the whiskers and outliers for each sex and target.

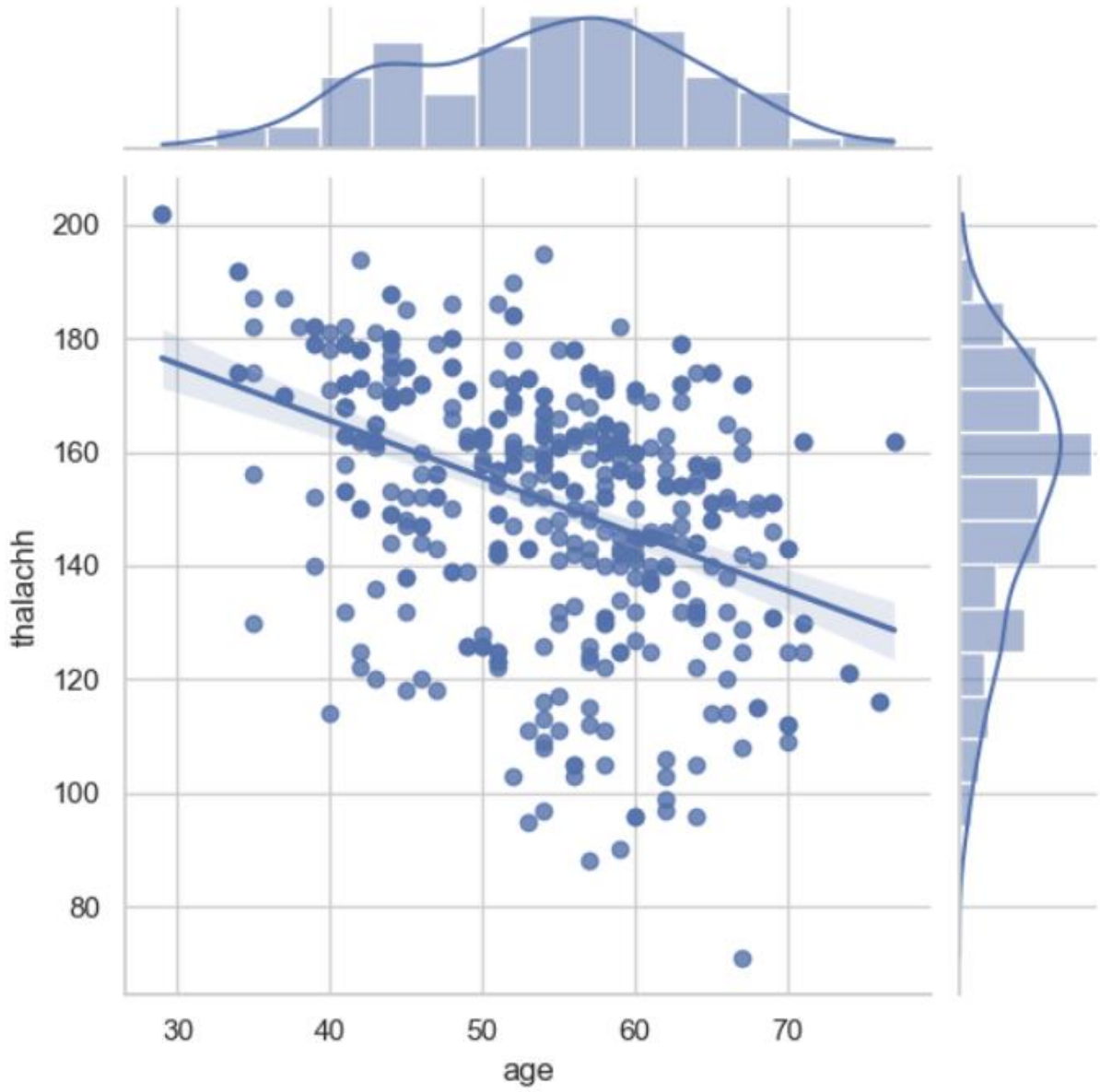
```
# Boxplot of Cholesterol by Gender and Heart Disease
plt.figure(figsize=(10, 6))
sns.boxplot(x='sex', y='chol', hue='target', data=data)
plt.title('Cholesterol Distribution by Sex and Heart Disease')
plt.show()
```



- Jointplot on relationships between thalachh and age.
- There is a negative correlation where when age increases, the maximum heart rate decreases.

```
sns.jointplot(x = 'age', y = 'thalachh', data = data, kind = 'reg')
# As the Age increases the heart rate is decreases
```

```
<seaborn.axisgrid.JointGrid at 0x27e80881090>
```

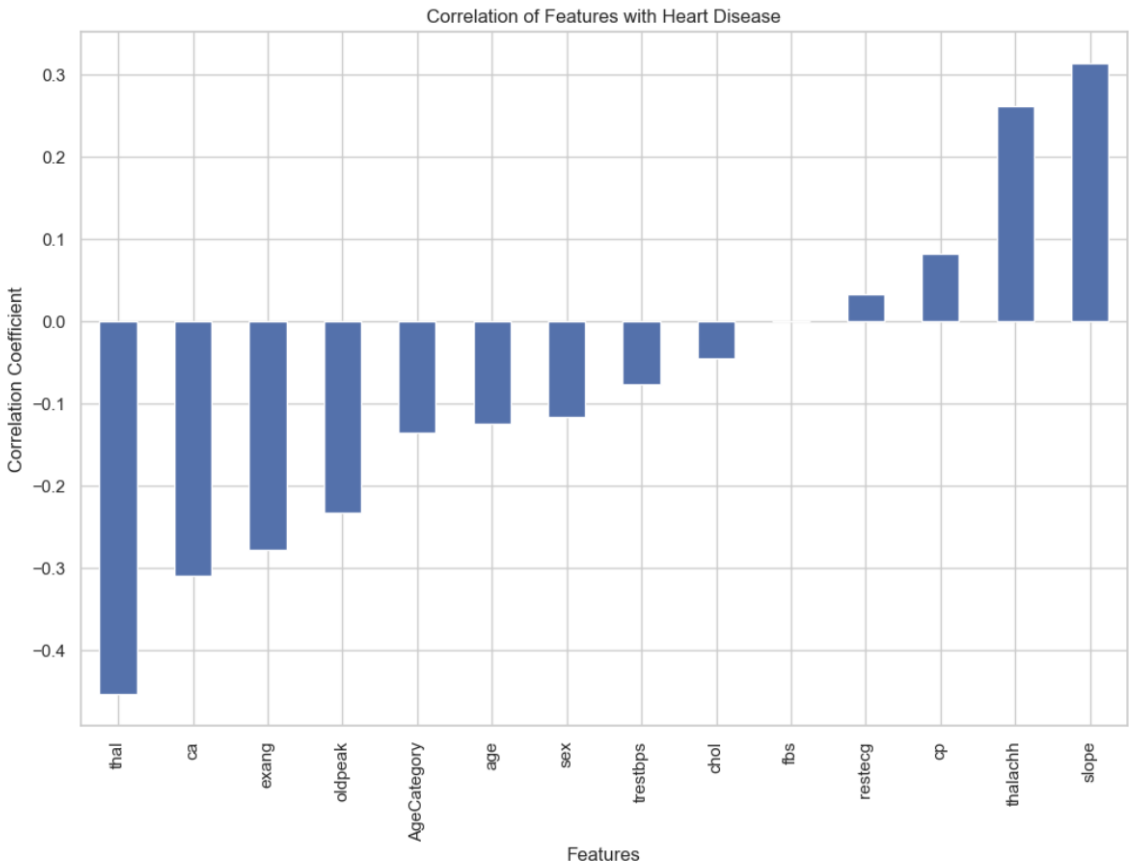


Discussion

Question 1

1. What are the most significant factors that contribute to heart disease?

```
# Question 1: Discussion
plt.figure(figsize=(12, 8))
data.corr()['target'].drop('target').sort_values().plot(kind='bar')
plt.title('Correlation of Features with Heart Disease')
plt.xlabel('Features')
plt.ylabel('Correlation Coefficient')
plt.show()
```

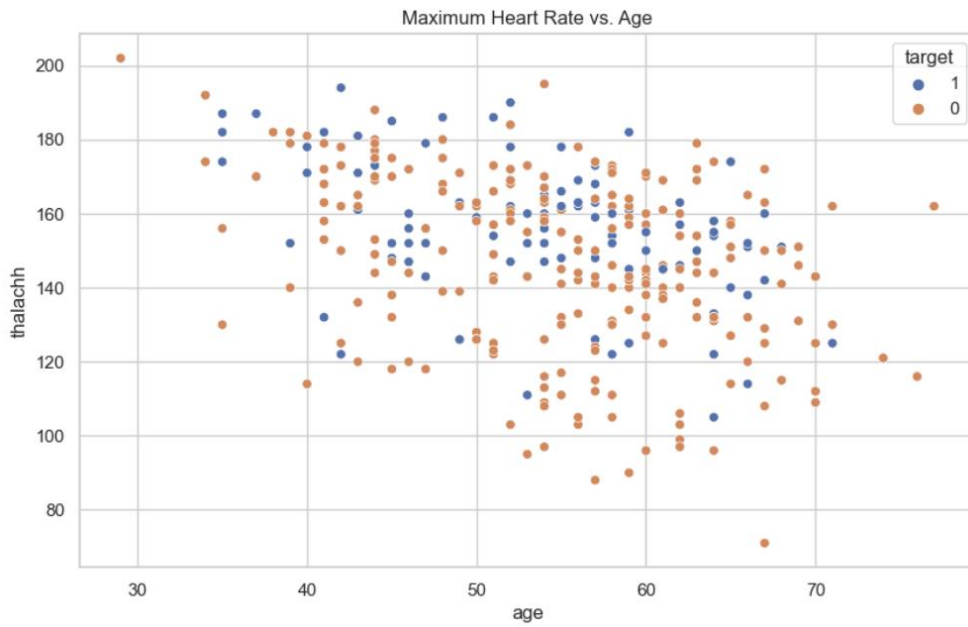


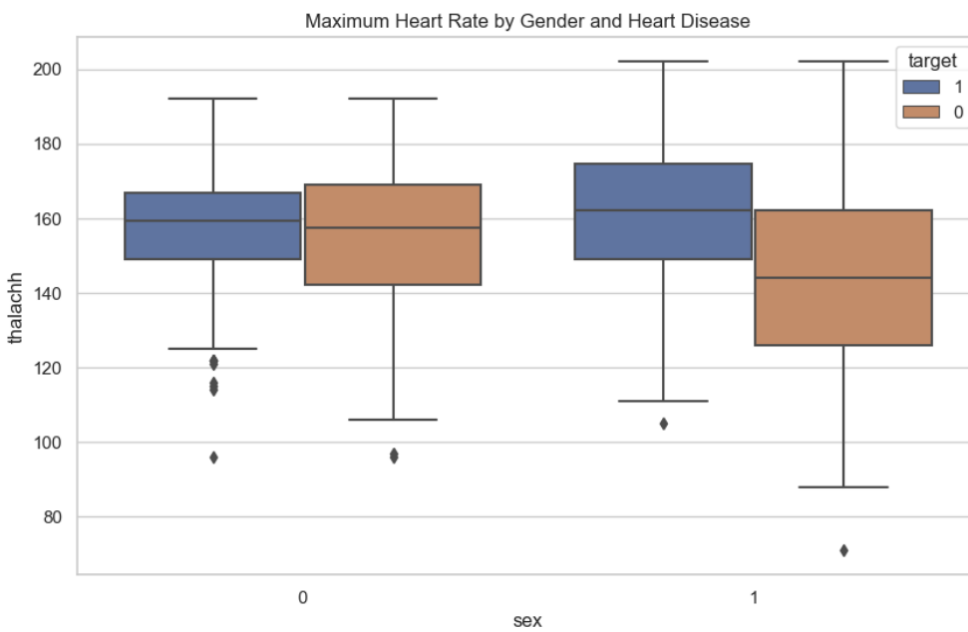
In the feature importance analysis of our predictive model, Thalassemia had the highest importance score, indicating a strong correlation with heart disease. This suggests that abnormalities in blood oxygen-carrying capacity play a crucial role in determining heart health risks. Other factors like maximum heart rate (Thalachh), cholesterol (Chol), and ST depression (Oldpeak) are also important, but Thalassemia stands out as the most influential.

Question 2

2. How does the maximum heart rate achieved (thalachh) vary across different age groups or genders?

```
# Question 2: Discussion
plt.figure(figsize=(10, 6))
sns.boxplot(x='sex', y='thalachh', hue='target', data=data)
plt.title('Maximum Heart Rate by Gender and Heart Disease')
plt.show()

plt.figure(figsize=(10,6))
sns.scatterplot(x='age', y='thalachh', hue='target', data=data)
plt.title('Maximum Heart Rate vs. Age')
plt.show()
```





We found out that younger individuals generally receive higher heart rates compared to older individuals due to the maximum heart rate (thalachh) are decreasing as the individuals aged. Individuals with heart disease (blue dots) have slightly higher heart rates than those without heart disease (brown dots) within the same age group.

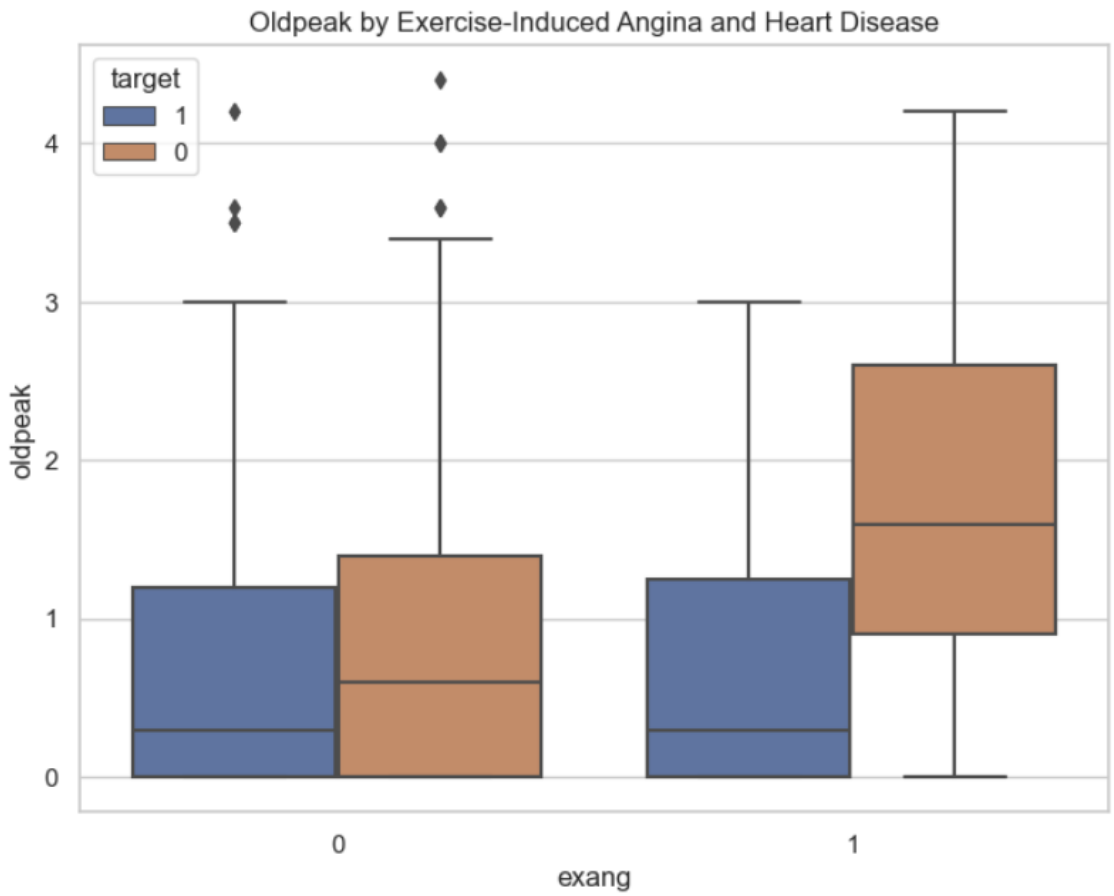
From the perspective of gender, females have a slightly higher median on maximum heart rate than males while males are having wider spread on maximum hear rates which indicates more variability. Individuals with heart disease (target=1, blue color) have a higher maximum heart rates compared to those without heart disease (target=0, brown color).

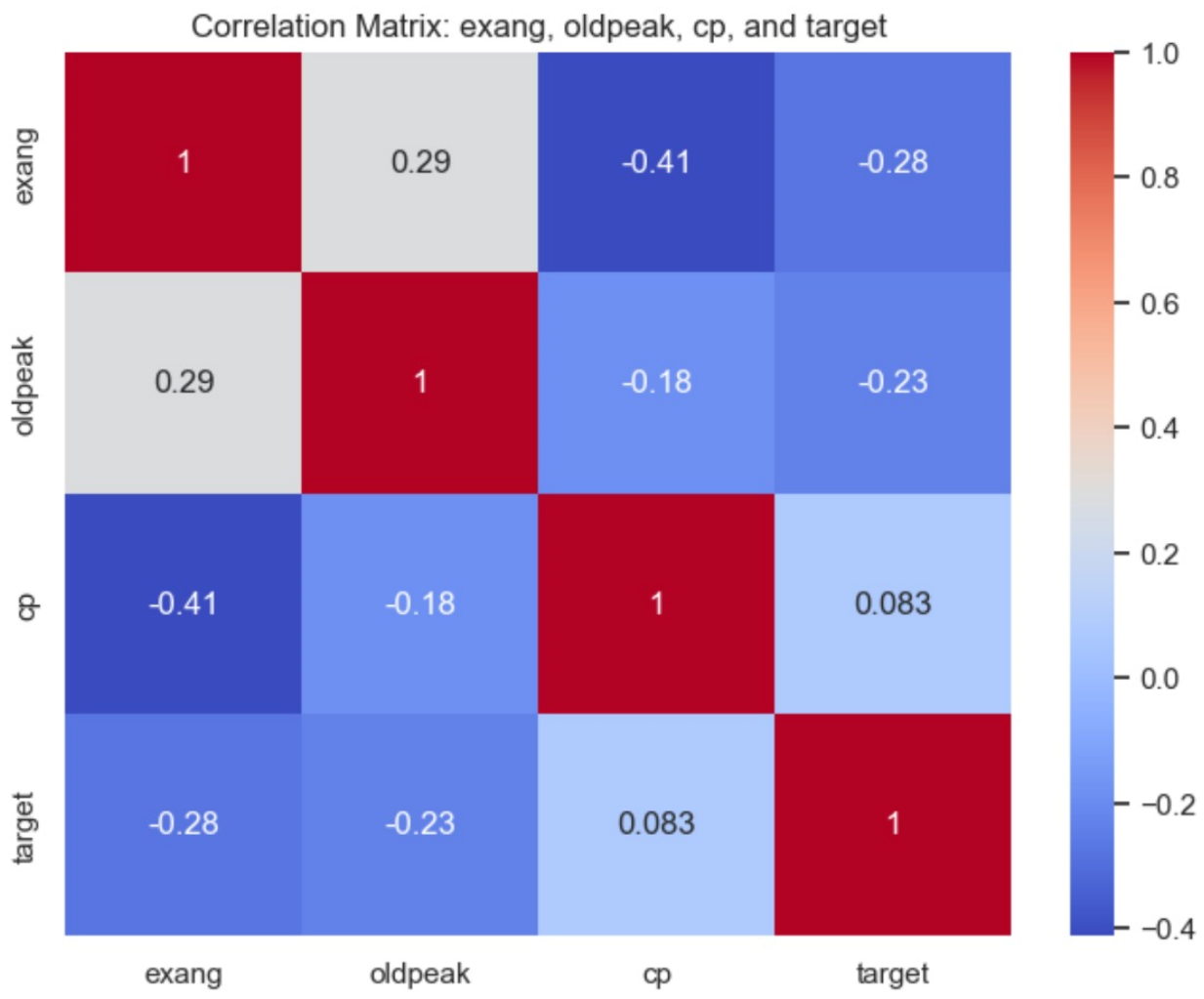
Based on the two graphs above, we can conclude that older males are more likely to have heart disease compared to younger individuals or females as heart disease are more likely to be observed in older individuals. With greater variability, older males tend to have a higher likelihood of heart disease.

Question 3

3. Is there a correlation between exercise-induced angina (exang) and other variables like oldpeak or cp?

```
# Question 3: Discussion
plt.figure(figsize=(8, 6))
sns.heatmap(data[['exang', 'oldpeak', 'cp', 'target']].corr(), annot=True, cmap='coolwarm')
plt.title('Correlation Matrix: exang, oldpeak, cp, and target')
plt.show()

plt.figure(figsize=(8, 6))
sns.boxplot(x='exang', y='oldpeak', hue='target', data=data)
plt.title('Oldpeak by Exercise-Induced Angina and Heart Disease')
plt.show()
```





Yes, there are correlation between exercise-induced angina (exang) and both variables like oldpeak or cp, but positive for oldpeak with a value 0.29, and negative value for cp with a value -0.41, as shown in the correlation matrix.While in the box plot, the result shows that those patients with exercise-induced angina (exang=1), they are more tended to have a higher oldpeak score, compared to those patients who does not have exercise-induced angina (exang=0).

Question 4

4. Can a predictive model accurately classify patients as having or not having heart disease based on these attributes?

```
# Question 4: Discussion
# Prepare the data
X = data.drop('target', axis=1)
y = data['target']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
# Initialize and train a RandomForestClassifier
model = RandomForestClassifier(random_state=42)
model.fit(X_train, y_train)
```

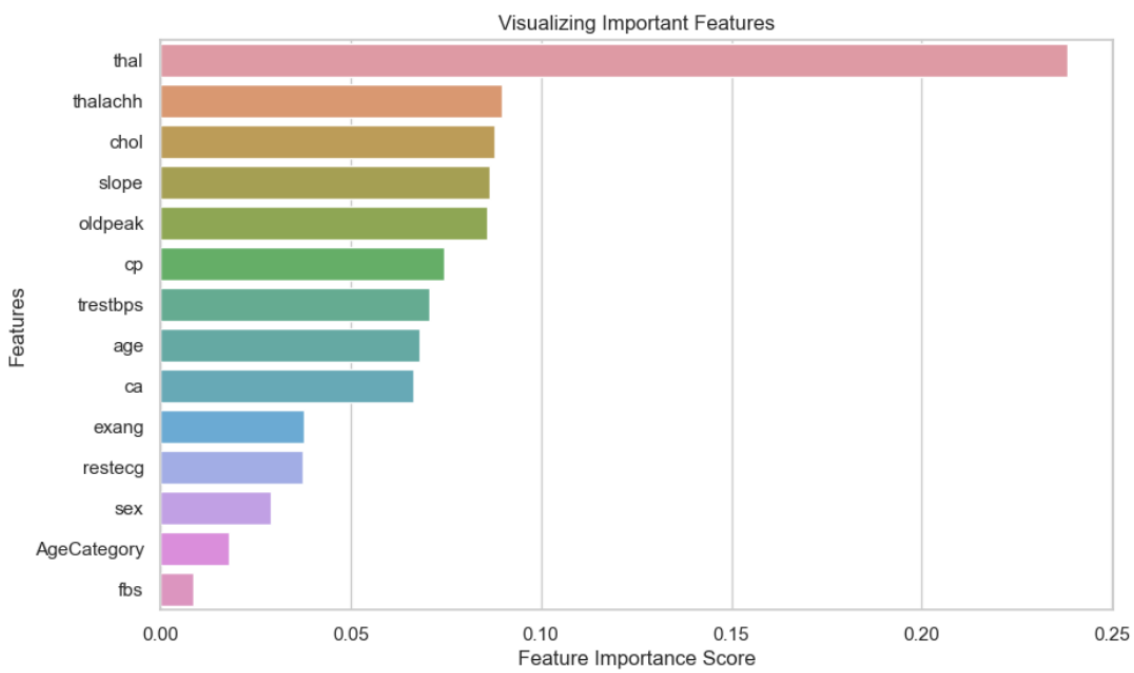
```
▾ RandomForestClassifier
RandomForestClassifier(random_state=42)
```

```
# Make predictions
y_pred = model.predict(X_test)
```

```
# Evaluate the model
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy}")
print(classification_report(y_test, y_pred))
print(confusion_matrix(y_test, y_pred))
```

```
Accuracy: 0.7976190476190477
              precision    recall  f1-score   support

           0       0.81      0.83      0.82        46
           1       0.78      0.76      0.77        38

    accuracy                           0.80        84
   macro avg       0.80      0.79      0.80        84
weighted avg       0.80      0.80      0.80        84

[[38  8]
 [ 9 29]]
```

```
feature_imp = pd.Series(model.feature_importances_, index=X.columns).sort_values(ascending=False)
plt.figure(figsize=(10,6))
sns.barplot(x=feature_imp, y=feature_imp.index)
plt.xlabel('Feature Importance Score')
plt.ylabel('Features')
plt.title('Visualizing Important Features')
plt.show()
```



Yes, the predictive model Random Forest Classifier accurately classifies patients for heart disease with an accuracy of 80% (0.7976). It shows a balanced performance based on the attributes. The model shows marginally high performance in identifying non-heart disease cases (class 0: precision=0.81, recall=0.83, f1-score=0.82) relative to heart disease cases (class 1: precision=0.78, recall=0.76, f1-score=0.77).

The feature importance graph shows that ('thal') Thalassemia is the most significant feature with a feature importance score of around 0.25. Other key features like ('thalachh') maximum heart rate, ('chol') cholesterol, ('slope') Slope of the peak exercise ST segment and (‘oldpeak’) ST depression induced by exercise relative to rest are features with high score.

Moreover, the confusion matrix shows that the model correctly identified 38 negative cases and 29 positive cases from the test set while producing 8 false positives and 9 false negatives. This shows the effectiveness of the chosen medical attributes in developing a predictive model for heart disease classification though opportunities for improvement beyond the existing 80% accuracy persist.

The findings of this investigation demonstrate why data analytics should be used in medical practice. Heart-related condition understanding as well as prediction capabilities increase significantly through data manipulation combined with exploration analysis and machine learning techniques. The model should receive future attention regarding accuracy optimization while also enabling the inclusion of added medical parameters to boost prediction reliability.

Conclusion

In conclusion, this exploration examines heart disease prediction data by performing data transformation and presentation tasks. A total of 1,888 patient records includes 14 medical attributes to understand what causes heart disease like age and blood pressure along with cholesterol levels and heart rate data.

We achieved accurate data through data preprocessing by addressing missing values together with outliers and inconsistent data. The study findings showed Thalassemia together with maximum heart rate and cholesterol levels and exercise-induced ST depression as the primary elements affecting heart disease. Our predictive model built using Random Forest Classifier reached 80% accuracy which demonstrates how machine learning technology excels at medical diagnosis.

The research demonstrates data-driven healthcare's significance alongside suitable data visualization methods which help identify diseases simultaneously with patient risk assessment and medical care determinations. Future research should concentrate on improving prediction accuracy along with implementing new medical data points to enhance forecasting capabilities.

References

Heart Disease Prediction Dataset. (n.d.). Retrieved February 6, 2025, from https://www.kaggle.com/datasets/mfarhaannazirkhan/heart-dataset/data